

Proyecto 1: Predicción de abandono de clientes (Churn)

Objetivo

Una empresa de suscripciones desea identificar **qué clientes tienen mayor probabilidad de cancelar su suscripción**, con el objetivo de contactarlos de forma preventiva y reducir la pérdida de clientes.

Para ello, se dispone de un conjunto de datos con **6.000 clientes**, incluyendo su historial y el registro de cuáles cancelaron el servicio. Con esta información se entrena un modelo de Machine Learning capaz de estimar la **probabilidad de que un cliente abandone la empresa (churn)**.

Parte 1: Construcción del Bundle

Se construye un **bundle** que contiene todos los elementos necesarios para realizar predicciones de manera consistente.

El bundle incluye los siguientes componentes:

- **Pipeline:** contiene el flujo completo del modelo de Machine Learning, incluyendo las transformaciones de datos y el modelo entrenado.
- **Columnas:** lista de variables de entrada que el modelo requiere para realizar una predicción.
- **Valores del plan:** conjunto de valores permitidos para la variable **Plan**.
- **Valores del método de pago:** conjunto de valores válidos para la variable **Método de Pago**.
- **Umbral:** valor utilizado para determinar cuándo un cliente se considera con riesgo de cancelar su suscripción.
- **Métrica principal:** se seleccionó **AUC (Area Under the ROC Curve)** como la métrica principal para evaluar el desempeño del modelo.
- **Metadatos:** información adicional como la versión de las librerías utilizadas, la fecha de entrenamiento y la versión del modelo.

```
# TAREA – Parte 1: armá el diccionario `bundle`

import sklearn
from datetime import datetime

bundle = {
    "pipeline": modelo,
    "columnas": list(X.columns),
    "valores_plan": list(clientes["plan"].unique()),
    "valores_metodo_pago": list(clientes["metodo_pago"].unique()),
    "umbral": 0.5,
    "metrica": f"ROC-AUC : {auc:.4f}",
    "metadata": {
        "pandas_version": pd.__version__,
        "numpy_version": np.__version__,
        "sklearn_version": sklearn.__version__,
        "joblib_version": joblib.__version__,
        "fecha_entrenamiento": datetime.now().strftime("%Y-%m-%d %H:%M"),
        "Modelo": "LogisticRegression"
    }
}

bundle
```

Python

Tarea 2: Persistencia y pruebas del modelo

En esta etapa se almacena el modelo entrenado en un archivo denominado:

modelo_churn.joblib

Posteriormente se desarrolla una función encargada de:

- Cargar el modelo.
- Recibir los datos de un cliente.
- Ejecutar la predicción.
- Devolver la probabilidad de abandono y la clasificación correspondiente.

```
# TAREA – Parte 2a: guardar el bundle en "modelo_churn.joblib"
#Guardar Bundle
joblib.dump(bundle,"modelo_churn.joblib")

# TAREA – Parte 2b: cargarlo de nuevo y predecir con `cliente_nuevo`.
# La probabilidad tiene que dar EXACTAMENTE la misma que arriba.

def predecir_cliente_nuevo(cliente: dict, bundle: dict) -> dict:

    fila = dict(cliente)

    X_nuevo = pd.DataFrame([fila])[bundle["columnas"]]

    probabilidad = bundle["pipeline"].predict_proba(X_nuevo)[0,1]

    return {
        "probabilidad": round(float(probabilidad),4),
        "Predicción": "CANCELA" if probabilidad >= bundle["umbral"] else "sigue activo"
    }
```

Python

Finalmente, se realizan pruebas utilizando dos perfiles de clientes:

- **Cliente con alto riesgo de abandono.**
- **Cliente fiel**, con baja probabilidad de cancelar el servicio.

Estas pruebas permiten validar el correcto funcionamiento del modelo antes de su publicación.

```
bundle_cargado = joblib.load("modelo_churn.joblib")

resultado = predecir_cliente_nuevo(cliente_nuevo.to_dict('records')[0],bundle_cargado)

print("Resultado via bundle",resultado)
```

Resultado via bundle {'probabilidad': 0.8813, 'Predicción': 'CANCELA'}

```
resultado = predecir_cliente_nuevo(cliente_fiel.to_dict('records')[0],bundle_cargado)

print("Resultado via bundle",resultado)
```

Resultado via bundle {'probabilidad': 0.0124, 'Predicción': 'sigue activo'}

Tarea 3: Desarrollo de la API

Para exponer el modelo como un servicio se implementa una API.

Esquemas

Se definen dos esquemas principales:

Esquema de solicitud (Request)

Contiene todas las variables necesarias para que el modelo pueda realizar la predicción de un cliente.

Esquema de respuesta (Response)

Devuelve la siguiente información:

- Probabilidad de abandono.
- Predicción del modelo.
- Nivel de riesgo del cliente.
- Umbral utilizado para la clasificación.

```

entrenar_modelo.ipynb × main.py U × esquema.py U ×
1 from pydantic import BaseModel, Field
2
3 class SolicitudClasificacion(BaseModel):
4     antigüedad_meses: int = Field(..., ge=0, description="Antigüedad del cliente en meses")
5     gasto_mensual: float = Field(..., ge=0, description="Gasto mensual del cliente")
6     visitas_ultimo_mes: int = Field(..., ge=0, description="Número de visitas del cliente en el último mes")
7     dias_desde_ultima_visita: int = Field(..., ge=0, description="Número de días desde la última visita del cliente")
8     tickets_soporte: int = Field(..., ge=0, description="Número de tickets de soporte abiertos por el cliente")
9     descuento_activo: int = Field(..., ge=0, le=1, description="Indica si el cliente tiene un descuento activo")
10    plan: str = Field(..., description="Plan del cliente (estandar, basico, premium)")
11    metodo_pago: str = Field(..., description="Método de pago del cliente (tarjeta, transferencia, efectivo)")
12
13 class RespuestaClasificacion(BaseModel):
14     probabilidad: float
15     prediccion: str
16     riesgo: str
17     umbral: float
18

```

Implementación

Posteriormente se desarrolla el archivo principal (**main**) encargado de iniciar la API.

Se implementan dos endpoints:

- **Endpoint de estado (Health Check):** permite verificar que la API se encuentra disponible y funcionando correctamente.
- **Endpoint de predicción:** recibe la información del cliente en formato JSON, utiliza el bundle previamente cargado para generar la predicción y devuelve el resultado al usuario.

```

@app.post("/predecir", response_model=esquema.RespuestaClasificacion)
def predecir(solicitud: esquema.SolicitudClasificacion):
    #validar modelo
    bundle = estado_servicio["bundle"]
    if bundle is None:
        raise HTTPException(status_code=503, detail="Modelo no cargado")

    fila = solicitud.model_dump()

    X_nuevo = pd.DataFrame([fila])[bundle["columnas"]]

    probabilidad = bundle["pipeline"].predict_proba(X_nuevo)[0,1]

    #devolver resultados

    return esquema.RespuestaClasificacion(
        probabilidad=round(float(probabilidad), 4),
        prediccion="Cancelación" if probabilidad > 0.5 else "No cancelación",
        riesgo="Alto" if probabilidad > 0.5 else "Bajo",
        umbral=0.5
    )

```

Pruebas de la API

Finalmente, se realizan pruebas funcionales de la API para verificar:

- La correcta recepción de los datos enviados por el cliente.

- La carga adecuada del modelo entrenado.
- La generación de predicciones consistentes.
- La estructura correcta de las respuestas en formato JSON.
- El correcto funcionamiento de todos los endpoints implementados.

Ciente en riesgo

Edit Value
Schema

```

{
  "antiguedad meses": 10,
  "gasto mensual": 14.0,
  "visitas ultimo mes": 4,
  "dias desde ultima visita": 26,
  "tickets soporte": 1,
  "descuento activo": 0,
  "plan": "basico",
  "metodo pago": "efectivo"
}

```

Code
Details

200

Response body

```

{
  "probabilidad": 0.8813,
  "prediccion": "Cancelación",
  "riesgo": "Alto",
  "umbral": 0.5
}

```

Response headers

```

content-length: 80
content-type: application/json
date: Sun, 06 Sep 2026 21:15:50 GMT
server: uvicorn

```

Ciente fiel

Edit Value
Schema

```

{
  "antiguedad meses": 20,
  "gasto mensual": 45.0,
  "visitas ultimo mes": 8,
  "dias desde ultima visita": 10,
  "tickets soporte": 0,
  "descuento activo": 0,
  "plan": "premium",
  "metodo pago": "tarjeta"
}

```

Code
Details

200

Response body

```

{
  "probabilidad": 0.0124,
  "prediccion": "No cancelación",
  "riesgo": "Bajo",
  "umbral": 0.5
}

```

Response headers

```

content-length: 83
content-type: application/json
date: Sun, 06 Sep 2026 20:41:17 GMT
server: uvicorn

```